Awaaz Flutter Mobile App

Project Handover Document

Document Version: 1.0

Prepared On: June 17, 2026

Application: Awaaz – Real Time City Alert

Package Name: eagle_eye

App Version: 1.0.13+14

Classification: Client & Developer Handover

Table of Contents

1. Project Overview

Project Name

Awaaz – Real Time City Alert (internal Dart package name: eagle_eye)

Purpose and Goals

Awaaz is a community-driven, location-aware mobile application that enables citizens to share real-time incident alerts, live video updates, and general community posts. The app helps users stay informed about events happening within a configurable radius (default ~10 km), interact through comments and reactions, chat with other users, and report or block inappropriate content.

Target Platforms

- Android (primary) – Application ID: com.awaazeye.cityalerts, minSdk 26, targetSdk 35
- iOS (primary) – Display name: Awaaz, CFBundleShortVersionString 1.2
- Web – Scaffold present (web/ folder, Firebase web config); not the primary delivery target
- Desktop (Windows/macOS) – Firebase options exist; not actively configured for production release

Key Features Summary

- Guest/device-based login and full authentication (email, Google, Apple Sign-In)
- Real-time news feed of incident and general posts with pagination
- Interactive Mapbox map showing nearby events
- Go Live – capture and publish live incident/general video alerts with categories
- Send Alert – targeted alerts to nearby users
- News details with comments, reactions, in-area related posts, and reporting
- User profiles, edit profile, friends, chat (Socket.IO), and blocked users
- Push notifications via OneSignal with deep-link navigation
- Branch.io deep linking for shared event URLs
- In-app support ticket system
- Settings: video quality, notification preferences, account deletion
- Google Mobile Ads with Firebase Remote Config-driven ad placement
- Firebase Analytics, Crashlytics, Performance Monitoring, and Remote Config

2. Technology Stack

- Flutter Version: 3.44.1 (stable channel)
- Dart Version: 3.12.1 (SDK constraint in pubspec.yaml: ^3.5.3)
- State Management: flutter_bloc (BLoC/Cubit pattern) with Freezed for immutable states
- Secondary utilities: Get package (^4.7.2) imported for limited use; primary navigation uses Navigator 1.0
- Networking: logic_go_network (private Bitbucket git dependency) + Dio
- Real-time: socket_io_client for chat and live updates

- Maps: mapbox_maps_flutter
- Local Storage: shared_preferences via PrefService wrapper
- Code Generation: build_runner, freezed, flutter_gen_runner

Key Packages and Purpose

- **flutter_bloc / bloc / freezed:** State management with immutable Cubit states
- **dio / logic_go_network:** REST API communication with typed APIType (public/protected)
- **socket_io_client:** Real-time chat, typing indicators, unread counts
- **mapbox_maps_flutter:** Interactive map with event annotations
- **firebase_core, firebase_auth, firebase_analytics, firebase_crashlytics, firebase_performance, firebase_remote_config:** Firebase platform services
- **onesignal_flutter:** Push notification delivery and click handling
- **google_sign_in / sign_in_with_apple:** Social authentication
- **flutter_branch_sdk / app_links:** Deep linking and share attribution
- **google_mobile_ads:** Monetization (banner, native, interstitial/open app ads)
- **image_picker, file_picker, camera, chewie, video_player, light_compressor_v2:** Media capture, playback, and compression
- **geolocator, location, geocoding, permission_handler:** Location services and permissions
- **flutter_screenutil:** Responsive UI scaling (design size 430×990)
- **connectivity_plus / internet_connection_checker:** Network connectivity monitoring
- **cached_network_image:** Optimized remote image loading
- **showcaseview:** In-app tutorial overlays
- **lottie:** Loading and success animations
- **youtube_player_flutter:** Embedded tutorial/help videos

3. Project Architecture

Architecture Pattern

The project follows a layered, feature-oriented architecture inspired by Clean Architecture principles but without a dedicated domain layer. The structure separates concerns into presentation (UI + Cubits), data (repositories + models + API), core (shared utilities, theme, widgets), and services (platform integrations). This is commonly referred to as a Presentation–Data–Core pattern with BLoC for state management.

Code Organization Strategy

- Feature modules live under lib/presentation/ grouped by flow (onboard, authentication, main)
- Each feature typically contains a screen widget and a bloc/ subfolder with Cubit + State (Freezed)
- API calls are centralized in lib/data/repositories/ (auth_repo.dart, main_repo.dart)
- Shared UI components reside in lib/core/widget/
- Global Cubits are registered once in lib/core/bloc_provider/bloc_providers.dart

- Routes are defined in lib/routes/app_routes.dart with onGenerateRoute
- Platform services (notifications, ads, sockets, camera) are isolated in lib/services/

4. Folder Structure Breakdown

lib/ – Root application source. Entry point: main.dart. Contains all Dart application code.

lib/core/ – Shared foundation: constants, theme, reusable widgets, utilities (PrefService, MapUtils, AppFunctions), and global BlocProvider registration.

lib/core/constant/ – App-wide constants: base URLs, event categories, report options, extensions.

lib/core/theme/ – ThemeData (darkTheme), color palette, and text styles.

lib/core/widget/ – Reusable UI components: buttons, text fields, loaders, dialogs, bottom sheets, video players.

lib/core/utils/ – Helper utilities: preferences, map helpers, skeleton loaders, hashtag formatter.

lib/core/bloc_provider/ – MultiBlocProvider configuration for all global Cubits.

lib/presentation/ – All UI screens organized by feature area: onboard/, authentication/, main/.

lib/data/ – Data layer: API controller, endpoints, models, and repositories.

lib/data/models/ – JSON-serializable data models (login, events, chat, profile, notifications, etc.).

lib/data/repositories/ – auth_repo.dart – authentication APIs; main_repo.dart – all other protected/public APIs.

lib/services/ – Platform and third-party integrations: OneSignal, Firebase, ads, sockets, camera, deep linking, connectivity.

lib/routes/ – app_routes.dart (route names + transitions), app_navigation.dart (Navigator helpers).

lib/gen/ – Auto-generated asset and font references (flutter_gen).

assets/ – Static assets: icons/, images/, videos/, fonts/, animation/ (Lottie JSON).

android/ / ios/ – Native platform configuration, permissions, signing, Firebase, Branch, and Mapbox setup.

test/ – Flutter test directory (minimal coverage – see Section 12).

lib/features/ – Not Available – features are organized under lib/presentation/ instead.

lib/domain/ – Not Available – no separate domain/use-case layer.

5. Setup & Installation Guide

Prerequisites

- Flutter SDK 3.44.x or compatible (Dart ^3.5.3)
- Android Studio / VS Code with Flutter & Dart plugins
- Xcode (macOS only, for iOS builds)
- Git access to Bitbucket for logic_go_network private dependency
- Firebase project access (aawaj-52af1)
- Mapbox access token
- OneSignal, Branch.io, and Google AdMob account credentials

Installation Steps

1. Clone the repository to your local machine.
2. Ensure Flutter is on PATH and run: flutter doctor
3. From project root, install dependencies: flutter pub get
4. Run code generation (Freezed / flutter_gen): dart run build_runner build --delete-conflicting-outputs
5. Place google-services.json (Android) and GoogleService-Info.plist (iOS) if not present.
6. Configure Mapbox token (see Environment Configuration below).
7. Connect a device or emulator and run: flutter run

Environment Configuration

- API Environment: Controlled by isLiveMode flag in lib/core/constant/app_constant.dart. true → https://awaazeye.com; false → http://139.59.32.181:8002
- Mapbox Access Token: Passed at build/run time via --dart-define=ACCESS_TOKEN=<your_token>. Read in main.dart via String.fromEnvironment('ACCESS_TOKEN'). iOS uses $(MAPBOX_ACCESS_TOKEN) in Info.plist.
- Firebase: Configured via lib/firebase_options.dart (generated by FlutterFire CLI).
- Branch.io Test Mode: Set io.branch.sdk.TestMode to false in AndroidManifest.xml for production (see README.md).
- No .env file is used; secrets and toggles are compile-time defines or in-code constants.

6. Build & Deployment

Android

- Debug APK: flutter build apk --debug --dart-define=ACCESS_TOKEN=<token>
- Release APK: flutter build apk --release --dart-define=ACCESS_TOKEN=<token>
- App Bundle (Play Store): flutter build appbundle --release --dart-define=ACCESS_TOKEN=<token>
- Release signing: android/app/build.gradle currently uses signingConfigs.debug for release – replace with production keystore before store submission.
- ProGuard/R8: minifyEnabled and shrinkResources enabled for release builds.

- ABI filters: armeabi-v7a, arm64-v8a only.

iOS

- Debug: flutter build ios --debug --dart-define=ACCESS_TOKEN=<token>
- Release: flutter build ios --release --dart-define=ACCESS_TOKEN=<token>
- Archive and upload via Xcode after configuring signing certificates and provisioning profiles.
- Set MAPBOX_ACCESS_TOKEN in Xcode build settings or xcconfig.

Web

Web scaffold exists. Build command: flutter build web. Not the primary production target.

Code Generation (CI/CD note)

Include dart run build_runner build --delete-conflicting-outputs in CI before flutter build.

7. State Management Explanation

The application uses the BLoC pattern via Cubit classes (flutter_bloc). States are immutable and defined with Freezed (@freezed) for copyWith, equality, and code generation. All primary Cubits are provided at the app root through MultiBlocProvider in main.dart.

Registered Global Cubits

- OnboardCubit, SplashCubit – onboarding and guest auth
- AuthScreenCubit, VerifyOtpCubit, ResetPasswordCubit, ChangePasswordCubit – authentication flows
- HomeScreenBlocCubit – bottom navigation and home shell state
- NewsScreenBlocCubit, NewsDetailsScreenBlocCubit – feed and detail views
- MapScreenCubit, GoLiveCubit, GeneralPostCubit, SendAlertCubit – map and content creation
- MyProfileCubit, EditProfileCubit, UserProfileCubit – profile management
- AlertsScreenBlocCubit, SearchScreenBlocCubit, SearchLocationCubit
- GetSupportCubit, NotificationSettingsCubit, DeleteAccountCubit, BlockedUsersCubit

Example Flow: News Feed (UI → Cubit → API → UI)

8. User opens HomeScreen → NewsScreen tab is first page in bottom navigation.
9. NewsScreen dispatches getNewsEvents('latest') on NewsScreenBlocCubit.
10. Cubit emits isLoading: true, then calls MainRepository.eventNewsList().
11. MainRepository uses logic_go_network RestClient with APIType.protected and Bearer token from PrefService.
12. API endpoint: GET api/v1/event-post/event-post-news?filterType=<type>&page=<n>&limit=<n>
13. Response parsed into EventNewsModel; Cubit emits updated eventPostList and pagination state.
14. NewsScreen rebuilds via BlocBuilder/BlocListener showing list or shimmer loader.
15. On 401 errors, AppFunctions.onTokenExpire() handles session expiry.

8. API Integration

API Structure

- Primary REST client: logic_go_network RestClient initialized in ApiController.init()
- Secondary Dio instance in ApiController for legacy/direct calls (base URL hardcoded separately)
- Repositories: AuthRepository (auth), MainRepository (all other endpoints)
- Response wrapper: ResponseModel { status, message, body }
- API versioning prefix: api/v1/

Base URL Handling

- Production: https://awaazeye.com
- Development: http://139.59.32.181:8002
- Socket URL: same host with trailing slash (baseSocketUrl in app_constant.dart)
- Toggle: isLiveMode boolean in app_constant.dart

Authentication Flow

16. Splash screen may perform guest login via device ID (api/v1/auth/guest-login).
17. Full auth supports email/password, mobile OTP, Google, and Apple sign-in.
18. On success, JWT access token stored in SharedPreferences (PrefService.accessToken).
19. Protected requests include Authorization: Bearer <token> via requestHeader(APIType.protected).
20. Token expiry (401) triggers AppFunctions.onTokenExpire() redirecting user to re-authenticate.

Key API Endpoint Groups

- Auth: login, register, OTP verify/resend, forgot/reset/change password, Google/Apple login
- User: profile CRUD, location/radius update, push token, block/unblock, delete account
- Events: post, draft, reactions, comments, search, map area events, view counts
- General posts: add post, draft
- Support: list, add support requests
- Notifications: user notification list, on/off toggles

Error Handling Strategy

- Repository methods catch logic_go_network Failure exceptions and parse error messages
- Errors returned as ResponseModel with status: false and user-facing message
- UI Cubits display errors via AppFunctions.showToast() or CommonSnackBar
- Connectivity checked via networkConnectionServices before critical operations
- Firebase Crashlytics captures unhandled Flutter errors when isLiveMode is true

9. Key Modules / Features

Splash & Onboarding

Description: App entry, guest device authentication, name/username/birthdate/profile setup.

Main Files: lib/presentation/onboard/splash_screen/, lib/presentation/onboard/onboard_screen/, lib/presentation/onboard/new_onboard_screen/

Flow: SplashCubit performs deviceAuth → stores token → navigates to Home or Onboarding screens based on profile completeness.

Authentication

Description: Email login/register, OTP verification, Google/Apple sign-in, password reset.

Main Files: lib/presentation/authentication/, lib/data/repositories/auth_repo.dart

Flow: AuthScreenCubit calls AuthRepository → saves token to PrefService → navigates to Home.

Home Shell

Description: Bottom navigation with News, Map, Go Live categories, and Profile tabs.

Main Files: lib/presentation/main/home/home_screen.dart, home_screen_bloc_cubit.dart

Flow: Initializes socket connection, permissions, push token, location updates, and showcase tutorial.

News Feed

Description: Paginated list of incident/general posts with filters (latest, trending, etc.).

Main Files: lib/presentation/main/news_screen/

Flow: NewsScreenBlocCubit fetches via MainRepository.eventNewsList with infinite scroll pagination.

News Details

Description: Full post view with video, comments, reactions, reporting, in-area related posts.

Main Files: lib/presentation/main/news_details_screen/

Flow: NewsDetailsScreenBlocCubit loads single post, comments, and handles user interactions.

Map

Description: Mapbox map displaying nearby event markers within user radius.

Main Files: lib/presentation/main/map_screen/, lib/core/utils/map_utils.dart

Flow: MapScreenCubit fetches area events and renders point annotations on Mapbox.

Go Live / General Post

Description: Camera-based live video recording, category selection, watermark, compression, and publish.

Main Files: lib/presentation/main/go_live_screen/, lib/presentation/main/general_go_live/, lib/services/camera_services.dart

Flow: GoLiveCubit / GeneralPostCubit upload via MainRepository.postEvent / generalAddPost.

Send Alert

Description: Send targeted alerts to users within radius.

Main Files: lib/presentation/main/send_alert/

Flow: SendAlertCubit handles alert composition and API submission.

Search

Description: Search events by hashtag and discover users by username.

Main Files: lib/presentation/main/search_screen/

Flow: SearchScreenBlocCubit queries search APIs with debounced input.

Chat & Friends

Description: Real-time messaging, friend requests, unread counts via Socket.IO.

Main Files: lib/presentation/main/chat_screen/, lib/presentation/main/add_friend_screen/, lib/services/socket_service/

Flow: SocketService connects with userId query param; events handled via SocketEvents constants.

Profile & Settings

Description: View/edit profile, notification settings, video quality, blocked users, account deletion.

Main Files: lib/presentation/main/my_profile_screen/, edit_profile_screen/, settings_screen/, delete_account_screen/

Flow: Profile Cubits call MainRepository for CRUD; settings persisted in PrefService.

Alerts / Notifications

Description: In-app notification list and push notification deep-link handling.

Main Files: lib/presentation/main/alerts_screen/, lib/services/one_signal_notification_service.dart

Flow: OneSignal click listener routes to news details via notification payload.

Support

Description: Submit and track support requests.

Main Files: lib/presentation/main/get_support_screen/

Flow: GetSupportCubit manages ticket list and creation via support APIs.

Ads

Description: Remote-config-driven ad placements (splash, news list native, banners).

Main Files: lib/services/ads_service/, lib/services/remote_config_service/

Flow: Firebase Remote Config provides test_ads_json; AdsService loads placements accordingly.

10. UI/UX Structure

Navigation

- Navigator 1.0 with named routes (MaterialApp.onGenerateRoute)
- Route definitions: lib/routes/app_routes.dart
- Navigation helpers: lib/routes/app_navigation.dart (NavigatorRoute class)
- Custom page transitions: fade, slide, scale, rotate via PageRouteBuilder
- Global navigatorKey: AppFunctions.navigatorKey for context-free navigation
- GoRouter / Navigator 2.0: Not used

Theme Management

- Single dark theme defined in lib/core/theme/app_theme.dart (darkTheme)
- Color palette: lib/core/theme/colors.dart
- Typography: lib/core/theme/text_styles.dart with Roboto, Test Tiempos Headline, and Alice fonts
- MaterialApp theme: darkTheme applied globally in main.dart

Responsive Design

- flutter_screenutil with design size 430×990 (portrait phone baseline)
- Text scaling locked to 1.0 via MediaQuery textScaler override
- Portrait-only orientation enforced in main.dart
- Shimmer skeleton loaders for loading states

11. Assets & Resources

Images & Icons

- assets/icons/ – SVG and PNG UI icons (search, filter, location, rescue, etc.)
- assets/images/ – PNG images (logos, category illustrations, placeholders)
- assets/videos/ – bundled video assets if any
- assets/animation/ – Lottie JSON (loader.json, success_animation.json)
- Generated references: lib/gen/assets.gen.dart (flutter_gen)

Fonts

- Roboto (Light 300 – Black 800) – assets/fonts/roboto/
- Test Tiempos Headline – assets/fonts/test_tiempos_headline/

- Alice Regular – assets/fonts/alice/
- Generated references: lib/gen/fonts.gen.dart

Localization

Not Available – No flutter_localizations or ARB files configured. The intl package is used for date/number formatting only. UI strings are hardcoded in English.

12. Testing & QA

- Unit Tests: Not Available – no unit test files beyond default scaffold
- Widget Tests: test/widget_test.dart exists but contains outdated counter-app boilerplate (will fail against current app)
- Integration Tests: Not Available
- Known Test Coverage: ~0% – no meaningful automated test coverage
- Manual QA recommended for: auth flows, go-live video upload, map rendering, push notifications, deep links, and payment-free ad flows

13. Performance Considerations

- Deferred initialization: heavy services (ads, crashlytics, camera) load in _deferredInit() after first frame
- cached_network_image for efficient image caching
- light_compressor_v2 for video compression before upload
- Pagination on news feed and notification lists to limit payload size
- Firebase Performance Monitoring integrated for production tracing
- Release builds: Android R8 minification and resource shrinking enabled
- Mapbox Impeller disabled on Android (EnableImpeller=false) for compatibility
- Socket reconnection with 5 attempts and 2s delay

14. Known Issues / Limitations

- Release Android build uses debug signing config – must be updated for Play Store production
- isLiveMode hardcoded to true – environment switching requires code change and rebuild
- ApiController Dio instance has a different hardcoded base URL (http://139.59.36.221:8001) than RestClient – potential inconsistency for Dio-only calls
- TokenInterceptor in ApiController is commented out
- Default widget test is obsolete and does not reflect the application
- No localization – English-only UI
- Web and desktop platforms not production-ready
- Some legacy package IDs in app_constant.dart (com.invoice.maker) appear unused/incorrect
- Branch.io TestMode must be manually verified before production release

- Guest login flow stores session without full profile – onboarding may be required afterward
- Legacy commented reference code from unrelated projects exists in lib/services/deep_linking.dart (BookClubLM/GetX) – should be removed

15. Future Improvements

- Introduce proper environment configuration (flavors or --dart-define) for dev/staging/prod
- Add domain layer with use cases to decouple Cubits from repositories
- Implement comprehensive unit and widget tests; fix existing widget_test.dart
- Migrate to GoRouter or Navigator 2.0 for declarative deep-link routing
- Add flutter_localizations and multi-language ARB support
- Replace hardcoded secrets with secure CI/CD secret injection
- Configure production Android signing and iOS provisioning in CI
- Consolidate ApiController Dio usage with RestClient or remove duplicate client
- Enable TokenInterceptor for automatic token refresh
- Improve error parsing in repositories (avoid brittle string splitting on Failure)
- Add integration tests for critical user journeys (login, post alert, receive notification)

16. Credentials & Access

IMPORTANT: Store all credentials securely. Values below are masked for handover documentation. Request full credentials from the project owner or secrets vault.

- Firebase Project ID: aawaj-52af1
- Firebase API Key (Android): AIzaSy********** (see firebase_options.dart)
- OneSignal App ID: 59df38e3-a27a-4f4c-bd65-9e19c39fbff7
- Google OAuth Client ID (iOS): 552605622755-**********.apps.googleusercontent.com
- Mapbox Access Token: Pass via --dart-define=ACCESS_TOKEN=pk.********
- Branch.io: Configured in AndroidManifest.xml and iOS Info.plist (app links: awaazeye.app.link)
- Google AdMob (test IDs in Remote Config defaults): ca-app-pub-3940256099942544/...
- Bitbucket logic_go_network: Requires authenticated git access to https://bitbucket.org/LogicGoNIkunj/logic_go_network.git
- Backend Admin Panel: Not part of this mobile codebase – separate admin web project may exist
- Admin Access Notes: Use backend-provided admin credentials for content moderation; not embedded in mobile app

17. Handover Notes

Important Developer Notes

- Always run build_runner after pulling changes that affect Freezed models or flutter_gen assets.
- Set isLiveMode correctly and verify Branch TestMode=false before any production release.

- Verify Firebase Crashlytics is recording errors only when isLiveMode is true.
- Mapbox token is mandatory for map functionality – builds without ACCESS_TOKEN will fail on map screens.
- Private git dependency logic_go_network must remain accessible for flutter pub get to succeed.
- Portrait orientation is enforced – test all screens in portrait only.

Things to Be Careful About

- Do not commit production keystore files or unmasked API secrets to version control.
- Changing baseUrl or isLiveMode affects both REST and Socket connections simultaneously.
- OneSignal and Branch deep-link handlers depend on app initialization order – notification clicks before init are queued in notificationPayload.
- Video upload flows depend on camera permissions, compression, and network – test on real devices.
- 401 token expiry triggers global logout – ensure refresh token flow is implemented before extending session duration.
- Remote Config ad JSON structure must match AdsJsonModel parsing – invalid JSON will break ad loading.

— End of Document —